PriorityQueue - Complete Notes

Simple revision notes written in the same way we discussed in chat.

This chapter explains PriorityQueue from zero to interview level with examples, diagrams, internal working, complexity, and common mistakes.

1. Why was PriorityQueue created?

A normal Queue follows FIFO: First In, First Out. But many real-life problems do not care about arrival order. They care about importance or priority. PriorityQueue was created for that kind of work.

Example: in a hospital emergency room, a serious patient should be treated before a less serious patient, even if the serious patient arrived later.

Main idea
Queue = arrival order
PriorityQueue = priority order
Use when importance matters more than insertion order

2. What is PriorityQueue?

PriorityQueue is a queue where the most important element is removed first. In Java, by default, the smallest element has the highest priority.

```
Example:

offer(30)
offer(10)
offer(20)

poll() returns 10 first
```

So even if 30 was inserted first, 10 can come out first because the queue is priority-based, not FIFO-based.

3. Real-life analogy

Think about a hospital emergency room. A patient with a minor fever may arrive first. Another patient with a heart attack may arrive later. The heart attack case should be treated first. That is exactly what PriorityQueue does.

Easy memory
Normal Queue: who came first?
PriorityQueue: who needs attention first?

4. Internal working

PriorityQueue does not use a LinkedList internally. It uses a Binary Heap, usually a Min Heap by default. The heap is stored in an array. The array is not fully sorted, but the smallest element stays at the top.

```
Conceptually:

          10
```

```
        /    \
      20      30
     /  \
   40    50

Smallest element stays at the top
```

This is why peek() can return the smallest element very quickly.

5. Min Heap idea

In a Min Heap, every parent is smaller than or equal to its children. So the root always holds the smallest value. PriorityQueue uses this idea by default.

Min Heap rule
Parent <= children
Root has the smallest element
peek() looks at root

6. offer() flow

offer() inserts a new element into the heap. Java first puts the new element at the end of the array and then restores the heap property by moving it upward if needed. This process is called heapify up.

```
Example:

Insert 30, 20, 10

Start:
30

Add 20:
30 20

Add 10:
30 20 10

Heapify up gives:
      10
     /  \
   30    20
```

Because the new element may need to move upward through the tree shape, offer() usually takes O(log n).

7. poll() flow

poll() removes the root, which is the highest-priority element. Then Java moves the last element to the root and restores the heap property by moving it downward. This process is called heapify down.

```
Example:

Before poll:
      10
     /  \
   30    20

Remove 10.
Move last element to root.
Heapify down:

      20
     /
```

```
    30
```

Since the root may move down through the heap, poll() usually takes O(log n).

8. peek()

peek() returns the root element without removing it. Because the root already holds the highest-priority element, peek() is O(1).

```
Example:

PriorityQueue:
      10
     /  \
   30    20

peek() -> 10
Queue stays the same
```

peek() is very fast because Java does not need to rearrange the heap.

9. Time complexity

Operation	Complexity	Why
offer()	O(log n)	Heapify up
poll()	O(log n)	Heapify down
peek()	O(1)	Root element
contains()	O(n)	Search one by one

10. PriorityQueue vs Queue

Queue	PriorityQueue
FIFO	Priority-based
First inserted removed first	Smallest element removed first by default
Common implementation: LinkedList	Internal structure: Binary Heap
Useful for waiting lines	Useful for importance-based processing

If the question says FIFO, think Queue. If the question says priority, think PriorityQueue.

11. Default smallest-first behavior

In Java, PriorityQueue uses natural ordering by default. For numbers, that means the smallest number comes first. So if you insert 5, 30 and 10, the first element removed will usually be 5.

```
Example:

offer(5)
offer(30)
offer(10)

poll() -> 5
poll() -> 10
poll() -> 30
```

This is why the order of insertion is not the same as the order of removal.

12. Custom priority using Comparator

If you want a different priority order, you can pass a Comparator. For example, if you want the largest element first, you can make the queue work like a Max Heap.

```
Example:

PriorityQueue<Integer> pq =
    new PriorityQueue<>((a, b) -> b - a);
```

Now the largest value gets the highest priority instead of the smallest one.

13. Common mistakes

A common mistake is thinking PriorityQueue keeps everything fully sorted. It does not. It only guarantees that the head is the highest-priority element. Another mistake is assuming it uses LinkedList internally. It does not; it uses a heap.

Another mistake is printing the queue and expecting sorted order everywhere inside. The internal heap order is not the same as a fully sorted list.

14. Useful methods

```
Common PriorityQueue methods:

offer(element)
poll()
peek()
remove()
contains()
size()
isEmpty()
```

These are the methods you should know for interviews and practical use.

15. Interview questions

Why was PriorityQueue created? Why is it not FIFO? Which element comes out first by default? What data structure does it use internally? Why is peek() O(1)? Why are offer() and poll() O(log n)? How do you make it work like a Max Heap?

16. Final revision sheet

Topic	Simple meaning
PriorityQueue	Queue based on priority
Default behavior	Smallest element first
Internal structure	Binary Min Heap
offer()	Add element and restore heap
poll()	Remove root and restore heap
peek()	Read root only
Queue comparison	Queue = FIFO, PriorityQueue = priority based
Custom order	Use Comparator

One-line memory: PriorityQueue = Binary Min Heap. Smallest element first by default. peek() is O(1), offer() and poll() are O(log n).